

MCDI500

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

AVANCE FASE 3 – SEMANA 2

SUMATIVA 2

Nacimientos en Chile entre 2020 y 2023

Integrantes: Cristian Hurtado, Jorge Altamirano, Renzo Vílchez y Luis Paredes

Docente: Dr. Omar Salinas Silva

Curso: MCDI500 – Programación para la Ciencia de Datos

Fecha: 29 de septiembre de 2026

Repositorio del proyecto: <https://github.com/jorgealtamiranonavarrete-art/proyecto-grupo2>

ÍNDICE

1. Introducción y alcance	3
2. Datos y transformaciones utilizadas.....	3
3. Núcleo orientado a objetos	4
4. Validación y resultados del tratamiento de talla	5
5. Eficiencia y elección del indicador.....	5
6. Reproducibilidad y evolución del repositorio	6
7. Conclusiones	6
Referencias	7

1. Introducción y alcance

Este informe presenta el núcleo algorítmico de la Sumativa 2 del proyecto sobre nacidos vivos inscritos en Chile entre 2020 y 2023. Amplía el avance formativo de F3 con clases que permiten comparar cuatro tratamientos de tallas ausentes y con pruebas de eficiencia en distintos tamaños. Las preguntas descriptivas y el archivo original se mantienen. Cada fila representa un nacido vivo inscrito, no necesariamente una madre o un parto distinto.

El trabajo parte del conjunto preparado en F2, con 735.611 filas. Su objetivo técnico es comprobar que una organización orientada a objetos reproduce exactamente los tratamientos anteriores y facilita su sustitución sin cambiar el cálculo. La guía de la Sumativa 2 pide integrar el pipeline anterior con el nuevo núcleo; el Apunte Fase 3 fundamenta la división funcional y la medición de eficiencia (Universidad Andrés Bello, 2026a, 2026b). La verificación independiente del código es relevante para la reproducibilidad de resultados científicos (Thimbleby, 2024).

2. Datos y transformaciones utilizadas

F2 documentó la fuente DEIS/MINSAL, los campos seleccionados, las conversiones de tipos, los valores ausentes y los códigos técnicamente inválidos de talla. Conservó el valor de origen, identificó las tallas observadas válidas y mantuvo todas las filas. La salida usada aquí contiene 597 tallas ausentes o inválidas para el análisis; no se borran nacimientos por ese motivo. F3 comprueba la cantidad de filas, años, meses, marcas derivadas, conservación de tallas observadas y huella SHA-256 del archivo original frente a los metadatos de F2.

Las cuatro alternativas de F2 son: no imputar; completar con la media general; completar con la mediana general; y completar con la mediana regional, usando la mediana general si una región no tiene tallas válidas. Tras comparar sus efectos sobre las medias y la dispersión, el grupo aprobó la mediana regional para tratar las tallas ausentes. Los metadatos de F2 registran esa decisión. La talla permanece en centímetros porque las preguntas descriptivas no requieren distancias ni escalamiento. La región se conserva legible para tablas y se muestra además como matriz one-hot en los dos notebooks de F3, aprendiendo categorías solo en entrenamiento (Pedregosa et al., 2011; scikit-learn developers, s. f.).

Verificación de F2	Resultado	Función en F3
Filas conservadas	735.611	Misma población de entrada
Años y meses	2020–2023; 1–12	Período esperado
Tallas observadas	735.014	No alteradas al preparar
Tallas imputadas	597	Coinciden con metadatos F2
SHA-256 del origen	Coincide	Mismo archivo documentado

3. Núcleo orientado a objetos

El módulo `F3/src/f3_estrategias_talla.py` contiene la clase abstracta `EstrategiaTalla`, cuatro clases concretas y `ComparadorImputacion`. La clase base exige ajustar y transformar; cada clase hija aprende y aplica un tratamiento diferente. El comparador recibe cualquiera de ellas y utiliza la misma llamada. Esta sustitución es un uso concreto del patrón Strategy, sin cambiar el resto del flujo. Las clases abstractas de Python permiten exigir métodos a las clases hijas (Python Software Foundation, s. f.-a).

La herencia aparece en las cuatro clases derivadas de `EstrategiaTalla`; el polimorfismo, cuando el comparador llama ajustar y transformar sin distinguir la clase; y el encapsulamiento, al mantener los valores aprendidos dentro del objeto. La propiedad `parametros` entrega una copia: modificarla desde fuera no altera la estrategia. Cada clase conserva una responsabilidad concreta y el notebook coordina las comprobaciones y la presentación de resultados

La operación común puede resumirse como `estrategia.ajustar(datos).transformar(datos)`. En esta comparación descriptiva se ajusta con los mismos registros completos de F2 para verificar equivalencia. Si en otra fase se construyera un modelo predictivo, los valores de imputación se aprenderían únicamente en entrenamiento y luego se aplicarían a prueba.

Componente	Responsabilidad
F2 y data/processed	Preparación, validaciones y conjunto legible de entrada.
F3/src/f3_algoritmos.py	Lectura, clase regional, dos cálculos y medición de tiempo y memoria.
F3/src/f3_estrategias_talla.py	Cuatro tratamientos de talla con una interfaz común.
F3/F3_Nucleo_Algoritmico.ipynb	Orquestación, pruebas, tablas y explicación de resultados.
F3/resultados/	Parámetros, versiones y mediciones pequeñas en JSON.

Como aporte de Jorge Altamirano a la revisión del equipo, el módulo `f3_algoritmos.py` incorpora `AnalizadorNacimientos`. Esta clase carga la salida preparada de F2 y ofrece los cálculos por bucle y por `groupby`. Ambos métodos reutilizan las funciones independientes, que permanecen disponibles para medir muestras de distintos tamaños. El notebook verifica la equivalencia de ambas formas de uso; la clase organiza el código, sin atribuirle una ganancia de tiempo que no se midió.

4. Validación y resultados del tratamiento de talla

Las cuatro series producidas por las clases se compararon fila por fila con la función original de F2 sobre los 735.611 nacimientos. La igualdad se verificó antes de interpretar resultados. Sin imputación quedan 735.014 tallas válidas; los otros tres tratamientos completan 597 valores y conservan las tallas observadas. La comparación prueba equivalencia técnica con F2; no demuestra que un valor imputado sea la talla real desconocida.

El notebook ejecutó ocho comprobaciones adicionales con datos pequeños: región sin talla válida, datos sin ausencias, región hipotética no vista al ajustar, copia externa de parámetros, transformación antes del ajuste, ausencia de una columna, ausencia de todas las tallas y talla negativa. Los casos inválidos producen errores explícitos. Los parámetros aprendidos, las versiones y estas comprobaciones quedan registrados en resultados/sumativa2_verificacion.json.

Método	Tallas válidas	Imputadas	Media (cm)
Sin imputar	735.014	0	48,905500
Media general	735.611	597	48,905500
Mediana general	735.611	597	48,905576
Mediana regional	735.611	597	48,905670

5. Eficiencia y elección del indicador

Se comparó el porcentaje regional calculado mediante un bucle de Python y mediante groupby de pandas. Para 10.000, 100.000 y 735.611 filas se verificó igualdad de resultados antes de medir. Cada operación se repitió tres veces con time.perf_counter() y se informa el menor tiempo de cada conjunto. En una ejecución separada se registró el pico de asignaciones rastreado por tracemalloc y el tamaño del resultado; medir memoria por separado evita alterar los tiempos. Las cifras son locales y pueden variar en otro equipo (Python Software Foundation, s. f.-b; The pandas development team, s. f.).

Filas	Bucle (s)	groupby (s)	Razón de tiempos
10.000	0,0027	0,0008	3,20
100.000	0,0205	0,0018	11,50
735.611	0,1420	0,0092	15,45

Filas	Pico bucle (MB)	Pico groupby (MB)	Entrada (MB)
10.000	0,016	0,339	0,09
100.000	0,158	2,789	0,86
735.611	0,017	21,748	6,31

En la ejecución completa, groupby fue 15,45 veces más rápido que el bucle, pero su pico de asignaciones rastreadas fue 21,75 MB frente a 0,017 MB del bucle. El bucle acumula conteos para 16 regiones; groupby crea estructuras intermedias para agrupar. tracemalloc no representa toda la memoria nativa de pandas ni el máximo del proceso, por lo que estas cifras no permiten afirmar el consumo total de cada método. Ambas salidas ocupan aproximadamente 0,0006 MB. Se conserva groupby para este indicador por equivalencia y ventaja temporal en el volumen completo, dejando explícito el mayor costo rastreado de memoria. El avance formativo documenta por separado la ventaja de leer solo seis columnas del CSV original.

6. Reproducibilidad y evolución del repositorio

El notebook F3_Nucleo_Algoritmico.ipynb reutiliza funciones de F2 y F3, comprueba la salida preparada de F2, muestra una matriz one-hot con vocabulario aprendido solo en entrenamiento y usa aplanar para registrar metadatos anidados. La recursividad se reserva para esa estructura de profundidad variable; no se usa para recorrer 735.611 filas. El cuaderno se ejecutó con Python 3.13.5, pandas 3.0.5, NumPy 2.5.2 y scikit-learn 1.9.0, conforme a requirements.txt, en ocho celdas de código consecutivas y sin errores.

El repositorio conserva F1, F2 y el avance formativo de F3. El desarrollo de la Sumativa 2 añade en F3 un módulo de estrategias, el notebook ejecutado y un registro pequeño de sus resultados; los README describen sus funciones y el orden de ejecución. Esta evolución separa lectura, cálculo, tratamientos, validación y documentación sin repetir la preparación de F2; así se puede contrastar el núcleo actual con el prototipo formativo y sus mediciones (Universidad Andrés Bello, 2026a, 2026b). En el historial aparece el merge 2f8f42d del 18 de septiembre registrado bajo la identidad externa «HR Informática» (HardyRojas). Esa cuenta no integra el Grupo 2 ni se le atribuye el desarrollo de los análisis. Se conserva la atribución original del merge y el archivo .mailmap normaliza solo los nombres de los cuatro integrantes.

7. Conclusiones

Las cuatro clases reproducen los tratamientos de F2 sin cambiar el dataset original ni los objetivos del proyecto. Su interfaz común permite agregar o sustituir tratamientos con menos cambios en el comparador. Las pruebas hacen visibles los casos esperados y los errores controlados. La comparación en tres tamaños respalda el uso de groupby para el indicador regional en este entorno, con una ventaja temporal y un mayor pico de memoria rastreada; ninguna cifra se presenta como constante universal. El notebook, el módulo y los JSON ofrecen evidencia verificable de este avance.

Referencias

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
<https://www.jmlr.org/papers/v12/pedregosa11a.html>
- Python Software Foundation. (s. f.-a). *abc* — *Abstract Base Classes*.
<https://docs.python.org/3/library/abc.html>
- Python Software Foundation. (s. f.-b). *time* — *Time access and conversions*.
<https://docs.python.org/3/library/time.html>
- scikit-learn developers. (s. f.). *OneHotEncoder*. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
- The pandas development team. (s. f.). *pandas.DataFrame.groupby*.
<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html>
- Thimbleby, H. (2024). Improving science that uses code. *The Computer Journal*, 67(4), 1381–1404. <https://doi.org/10.1093/comjnl/bxad067>
- Universidad Andrés Bello. (2026a). *Guía de apoyo — Evaluación Sumativa 2 (Fase 3): Núcleo algorítmico, eficiencia e implementación orientada a objetos* [Material de curso]. MCDI500.
- Universidad Andrés Bello. (2026b). *MCDI500: Apunte Fase 3* [Material de curso]. MCDI500.